REPUBLIQUE DU CAMEROUN

PAIX – TRAVAIL – PATRIE

MINISTERE DE L'ENSEIGNEMENT SUPERIEURE

REPUBLIC OF CAMEROON

PEACE – WORK – FATHERLAND

MINISTRY OF HIGHER EDUCATION

FACULTY OF ENGINEERING AND TECHNOLOGY

PO BOX 63

BUEA, SOUTH WEST REGION

DEPARTMENT OF COMPUTER ENGINEERING

COURSE CODE: CEF440

COURSE TITLE: INTERNET PROGRAMMING AND MOBILE DEVELOPMENT

COURSE INSTRUCTOR: Dr. NKEMENI VALERY

TASK 5 REPORT: User Interface (UI) Design and Frontend Implementation

Presented by:

NAMES	MATRICLE
1. **MEKOLLE ASHLEY ARREYMANYOR**	**FE22A247**
2. **AKO RUTH ACHERE**	**FE22A144**
3. **NGUIMENANG ZEUFACK KEREINE**	**FE22A266**
4. **TUMUTOH LYDIE-KASEY BIHNUI**	**FE22A321**
5. **TIWA DELAN NDIKA**	**FE22A319**

MAY 2025

Table of Contents

1. UI Design and Implementation Document

1.1. Introduction

1. Introduction

Task 5 of this project focuses on the **UI Design and Frontend Implementation** of the *SmartCheck* mobile application. This stage is essential because it transforms conceptual system requirements into a **functional, visual, and interactive experience** for users—primarily students and lecturers. The objective was to develop a clean, user-friendly interface that supports key attendance workflows while also showcasing the app's branding, modular structure, and biometric capabilities.

The *SmartCheck* system addresses long-standing challenges in academic attendance tracking by replacing manual or semi-digital methods with a solution that combines **facial recognition and geofencing**. For such a system to succeed, the frontend must not only be functionally complete but also visually intuitive, responsive, and fast.

Throughout this phase, design and development efforts were guided by principles of:

- **Simplicity**: Interfaces that are clean and uncluttered, ensuring minimal user confusion.
- **Consistency**: Shared components and a uniform style guide across all screens.
- **Reusability**: Modular design patterns that make the app scalable and maintainable.
- **Accessibility and Responsiveness**: Layouts that adapt smoothly across various device sizes and user needs.

The implementation was done using **React Native CLI** with JavaScript, leveraging reusable components, stack navigation, and third-party libraries for camera and GPS access. This ensures that the system runs efficiently on Android devices, offers smooth user flows, and remains flexible for future enhancements.

This report outlines the design identity of the app, visual design decisions, reusable frontend components, user experience (UX) considerations, the overall modular architecture used to build the system interface, and a few UI build using Figma

1.2. App Identity

1.2.1. App Name: SmartCheck

The application is titled **SmartCheck**, a concise and intuitive name that reflects the primary purpose of the system: **smart, secure, and location-aware attendance checking**. The name reinforces trust and precision, qualities that are essential in an academic environment where attendance data must be both reliable and tamper-proof.

- The term "Smart" highlights the app's use of **advanced technologies** like **facial recognition** and **geofencing**,
- While "Check" represents the central function of confirming student presence. Together, SmartCheck communicates a digital transformation of traditional attendance systems.

1.2.2. Tagline: "Where Presence Meets Precision"

The tagline successfully expresses the mission of the application.

- **Presence** refers to physically verifying that a student is within the classroom geofence.
- **Precision** emphasizes the biometric verification step through facial recognition.

This phrase highlights the **dual validation process** (location + face), making the system **accurate, secure, and fair**.

1.2.3. Purpose

SmartCheck was designed to eliminate the inefficiencies of manual and semi-digital attendance methods, such as:

- Proxy attendance (someone signing in for another)
- Time-consuming roll calls
- Incomplete or unreliable records

Its core purpose is to:

- Provide an automated attendance management solution for students and lecturers
- Use facial recognition to uniquely identify students
- Apply geofencing to ensure students are physically present within a defined classroom location.

By integrating these two technologies, SmartCheck ensures attendance is **quick, secure, and fraud-resistant**, while also offering **real-time tracking and visibility** for both students and instructors.

1.2.4. Logo

Fig1: SmartCheck Official Logo

The SmartCheck logo features a dynamic and colorful design centered around a **checkmark**, symbolizing completion, approval, and attendance confirmation. The checkmark is surrounded by **radiating droplets in multiple colors**, giving it a vibrant and modern appeal.

- The checkmark reinforces the core functionality—smart check-in.
- The multiple colors reflect diversity, adaptability, and inclusivity, making the system visually welcoming and scalable to various institutions.

Visually, the logo strikes a balance between **professionalism and simplicity**, aligning with the app's objective to remain clean and intuitive for users.

1.2.5. Audience

The SmartCheck application is targeted primarily at:

- **Students**: for quick and secure check-in to lectures
- **Lecturers**: to monitor attendance, initiate sessions, and resolve disputes
- **IT Administrators**: to manage system-wide configurations, data, and maintenance

These users span across higher education institutions such as universities, polytechnics, and professional training centers.

The UI and UX design have been crafted to ensure ease of use for both **t**ech-savvy users and those with minimal mobile experience.

1.2.6. Color Palette

- **Primary Blue (#007BFF)**
   signifies Trust, reliability, and intelligence
- **Secondary Colors:** Red, Yellow, Green, Purple represents Visual inclusiveness, action highlights, and diversity. These colors are echoed in the logo droplets and used subtly in button styles, tabs, and indicators throughout the app.
- **Typography:**
  - Clean sans-serif fonts for readability and modern appearance
  - Emphasis on **bold section headers** and **light body text**
- **Tone and Feel:**
  - Professional but friendly
  - Simple, lightweight, and uncluttered screens
  - Visual hierarchy for quick comprehension (e.g., large icons, bottom navigation, white space)
  - Blue-and-white theme for a clean, calm launch experience

1.3. Branding Philosophy

SmartCheck is more than just an attendance app, it is a digital solution engineered to reflect the core values that academic institutions seek to uphold: operational efficiency, technological innovation, and data integrity. These three pillars are not only visible in the app's functionality but are also deeply embedded in the **visual design, user interactions, and system architecture.**

1.3.1. Efficiency

Efficiency is at the heart of SmartCheck's branding and system experience. The goal is to make attendance taking as fast, frictionless, and automated as possible while reducing human workload and error.

1.3.1.1. How branding reflects efficiency:

- **Quick check-in process**: Students can check in under 5 seconds using facial recognition and geolocation validation, replacing manual sign-in sheets and roll calls.
- **Minimalist UI**: The interface is designed to be clean and uncluttered, with only essential options on screen, reducing cognitive load.

- **Reusable UI Components** like Header, Footer, and Card maintain consistency, allowing users to instantly recognize functions and navigate quickly.
- **Color coding**: Blue as the primary color signifies calmness, serenity and reliability which helps users feel confident and safe in the app's responsiveness.

Efficiency ensures higher adoption rates among lecturers and students and makes the system practical for use in real-time classroom environments.

1.3.2. Innovation

SmartCheck embodies innovation through the use of advanced mobile technologies to solve real world academic challenges.

1.3.2.1. How branding reflects innovation:

- **Modern name and logo**: The name "SmartCheck" and its checkmark symbol are forward-looking, while the splash of colors adds a tech-savvy, dynamic feel.
- **Technology-driven process**: Use of facial recognition and geofencing to verify real presence. This innovation distinguishes SmartCheck from typical attendance tools.
- **React Native implementation**: A modern cross-platform mobile development framework ensures flexibility and scalability for future innovations.

Branding the app around innovation communicates to institutions and users that SmartCheck is not just a basic utility, it's a **next-gen attendance solution** ready for future integrations (e.g., biometric logs, AI reports, IoT classrooms).

1.3.3. Integrity

In any academic environment, **honesty and transparency** in attendance records are vital. SmartCheck is built to ensure **accountability**, both technically and visually.

1.3.3.1. How branding reflects integrity:

- **Dual-layer validation**: Combining facial recognition with GPS location ensures that students are physically present and personally identified, preventing impersonation.
- **Transparent reporting**: Admins can view detailed attendance logs, complete with timestamps and status updates.
- **Leave/Dispute features**: Students can formally file requests, while lecturers can approve/reject them. This feature supports fairness

Integrity in branding helps SmartCheck build trust with users and ensures that both administration and students view the app as a fair and unbiased system.

1.4. Frontend Implementation

The frontend of the **SmartCheck** application was developed using **React Native CLI**, applying modern software engineering principles such as **modularity**, **component reusability**, and a focus on **user experience (UX)**. This ensures the system is easy to scale, maintain, and use across various mobile devices.

1.4.1. Technology Stack

The frontend of **SmartCheck** was built using a combination of modern, flexible, and widely adopted mobile development tools to meet the system's performance, usability, and scalability requirements. The selected technologies enable modularity, cross-platform compatibility, and integration with core device features such as the camera and GPS.

1.4.1.1. React Native CLI (JavaScript)

The SmartCheck frontend was developed using **React Native CLI**, a framework that allows developers to build native mobile applications using JavaScript. Unlike Expo, which abstracts some native configurations, React Native CLI provides full control over lower-level device APIs—critical for features like facial recognition, location services, and offline handling.

React Native's component-based architecture also supports the creation of reusable UI elements, such as custom buttons, input fields, headers, and footers, which contribute to consistent styling and performance across screens.

1.4.1.2. JavaScript (ES6+)

The entire codebase was written in **plain JavaScript** (no TypeScript), keeping the syntax lightweight and beginner-friendly while still leveraging ES6+ features like arrow functions, destructuring, async/await, and modular imports. This makes the app easy to maintain and accessible to all group members regardless of experience level.

1.4.1.3. React Navigation (Native Stack)

The app uses ***@react-navigation/native*** and ***@react-navigation/native-stack*** to manage screen transitions and stack-based routing. Each page (e.g., Home, Attendance Login, Facial Recognition, Attendance History) is registered in a navigation stack and dynamically rendered based on user actions. Navigation is centralized in App.tsx for better maintainability.

This setup ensures that users can move smoothly between screens, return to previous steps, or access core features like disputes and reports from any context within the app.

1.4.1.4. Camera Module – react-native-camera

To implement facial recognition, the app integrates ***react-native-camera,*** which provides access to the device's front-facing camera. This module supports:

- Real-time face capture

- Custom camera views and bounding boxes
- Access to image data (base64 or URI) for backend verification

The camera is optimized for fast access and minimal latency to improve the user experience during check-in.

1.4.1.5. Geolocation – react-native-geolocation-service

For geofencing functionality, the app uses react-native-geolocation-service, a more reliable alternative to the default React Native Geolocation API. It offers:

- Access to device GPS and network-based location
- Support for both Android and iOS
- High accuracy mode and background permission prompts

GPS coordinates are used to verify whether the student is physically within the defined geofence boundary of the classroom before allowing check-in.

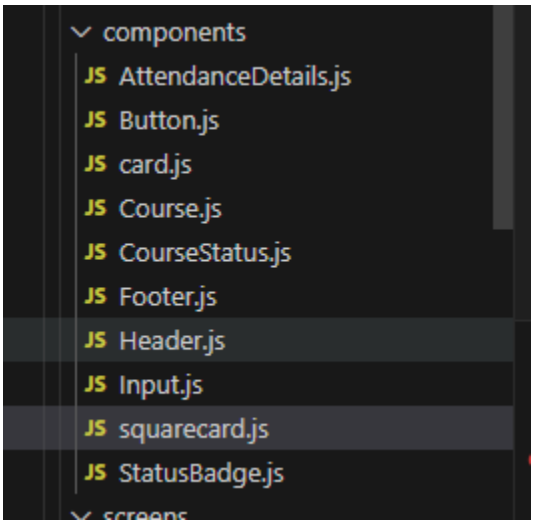
1.4.1.6. Icon Library – react-native-vector-icons (Ionicons)

All icons used in the app (for navigation, actions, and interface indicators) come from **Ionicons**, one of the most complete icon sets in the mobile development ecosystem. Icons help reduce textual clutter and guide user interaction through visual symbols.

1.4.2. UI Framework – Custom Components

Instead of using a heavy UI library, SmartCheck uses custom reusable **components** such as:

- Header.js: Unified top navigation bar with title and notifications
- Footer.js: Bottom tab bar with active tab highlights
- SquareCard.js: Clickable dashboard tiles for key actions
- Input.js, Button.js, Card.js: Standardized UI blocks for forms and lists



These components are responsive, easy to style, and highly maintainable, contributing to a clean, lightweight user interface.

1.4.2.1. State Management – React useState and Props

For this version of SmartCheck, local component state and props were sufficient to manage UI behavior and inter-screen communication. Features like:

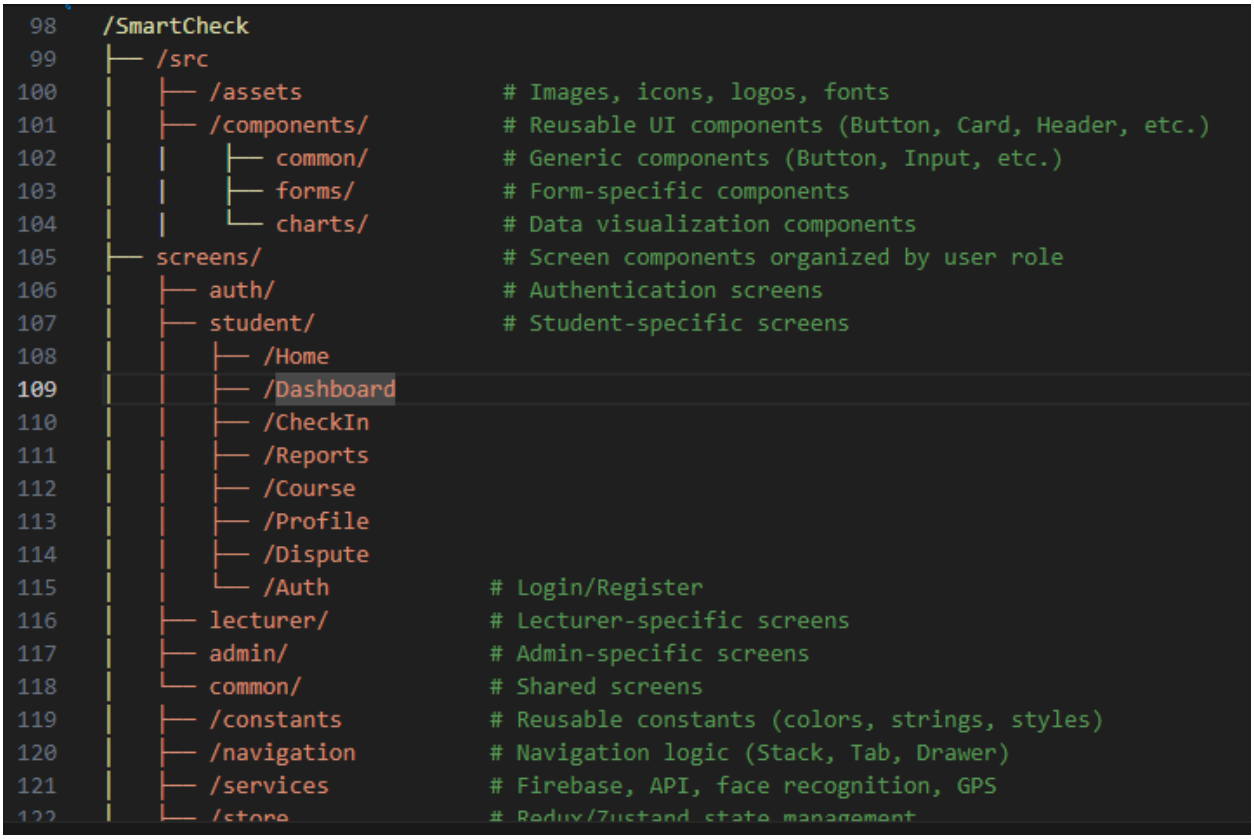
- Active tab tracking
- Check-in status
- Notification counts
- Form inputs and face capture state

are all handled using **React Hooks**, specifically useState. This keeps the state predictable and tightly scoped to the components that need it.

1.4.3. Modular Architecture and File Structure

The SmartCheck application adopts a **modular architecture**, where related features and components are logically grouped into **independent folders**. This approach enhances **code readability**, **reusability**, and **team collaboration**, making it easier to maintain and scale the system as features grow.

Each module is built as a **self-contained unit**—with its screens, components, and styles—minimizing dependencies and simplifying debugging.



1.4.3.1. Key Structure Principles:

- **Feature-based organization**: Files are grouped by *what they do*, not by *file type*, which makes navigation intuitive.
- **Shared components** (e.g., Header, Footer, Button) are placed in a central components folder and reused across all screens.
- **Screens are grouped by functionality**: For example, check-in screens like AttendanceLogin.js and FacialRecognition.js are stored under screens/CheckIn/.

Each screen has its **own folder and JS file**, making it easier to debug, test, and enhance specific features without interfering with others.

1.5. Reusability and Modularity

Reusability was a core focus in the UI design:

1.5.1.1. Reusable Components:

- **Header.js**: Displays the app name, notification icon, and badge count across all screens.
- **Footer.js**: Provides consistent bottom navigation with active tab highlighting.
- **SquareCard.js**: Used for navigation cards on the home screen.
- **Button.js / Input.js**: Styled UI elements reused in login, dispute forms, etc.

1.5.1.2. Benefits:

- Reduces duplicate code
- Enhances maintainability
- Enforces UI consistency across modules

Example:

The same SquareCard.js is used to display “Check-In”, “View Courses”, and “Create Dispute” options on the Home Screen—ensuring uniform style and behavior.

1.6. Navigation System

The SmartCheck application uses a **stack-based navigation system** implemented with **React Navigation**, one of the most widely adopted libraries in React Native. This navigation structure allows smooth transitions between screens and maintains a predictable history of user interactions.

1.6.1. Navigation Structure Overview

SmartCheck uses **Native Stack Navigation**, where each screen is registered in a central navigator (App.tsx) and loaded dynamically as users interact with the app.

```
// Import your JavaScript screens
import HomeScreen from './src/screens/Home/Home';
import AttendanceLoginScreen from './src/screens/CheckIn/AttendanceLogin';
import FacialRecognitionScreen from './src/screens/CheckIn/FacialRecognition';
import Signup from './src/screens/Aut/Signup.js';
import Login from './src/screens/Aut/Login.js';
import AttendanceHistory from './src/screens/CheckIn/AttendanceHistory';

const Stack = createNativeStackNavigator();

Tabnine | Edit | Explain
const App = () => {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Signup" screenOptions={{ headerShown: false }}>
        <Stack.Screen name="Signup" component={Signup} />
        <Stack.Screen name="Login" component={Login} />
        <Stack.Screen name="Home" component={HomeScreen} />
        <Stack.Screen name="AttendanceLogin" component={AttendanceLoginScreen} />
        <Stack.Screen name="FacialRecognition" component={FacialRecognitionScreen} />
        <Stack.Screen name="AttendanceHistory" component={AttendanceHistory} />
      </Stack.Navigator>
    </NavigationContainer>
  );
};

export default App;
```

This system allows:

- Transitioning between different screens (e.g., from Home to Check-In)
- Passing data between pages (e.g., user status or session ID)
- Managing backstack behavior (e.g., returning from camera to login)

1.6.1.1. Navigation Behavior

- Navigation is **centralized** in App.tsx, which ensures consistency and scalability.
- All navigation flows are **unidirectional** and returnable (stack behavior)
- The **initial screen** is set to "Opening" using:

1.7. Key Screens Implemented

1.7.1. User Experience (UX) Considerations

The app was designed with **end-user simplicity and usability** in mind:

1.7.1.1. For Students:

- One-click access to essential features (check-in, history, disputes)
- Step-by-step attendance flow with live feedback
- Clean, readable typography and color-coded actions

1.7.1.2. For Lecturers:

- Quick access to course sessions and bulk check-ins
- Leave/dispute management interface

1.7.1.3. Mobile Responsiveness:

- All layouts tested across multiple screen sizes
- ***ScrollView*** used where necessary to maintain accessibility on small screens

1.7.1.4. UX Design Principles Applied:

- Clarity over complexity
- Action-first UI
- Feedback for every interaction
- Error prevention (e.g., facial scan cannot proceed unless location is valid)

1.7.2. Major UI for SmartCheck

The SmartCheck application consists of a carefully selected set of key screens that align directly with the system's core functionalities: biometric attendance validation, geofencing, academic record access, and dispute handling. Each screen is designed with simplicity and clarity in mind, using reusable components and maintaining visual consistency.

1.7.2.1. Signup and Login Pages

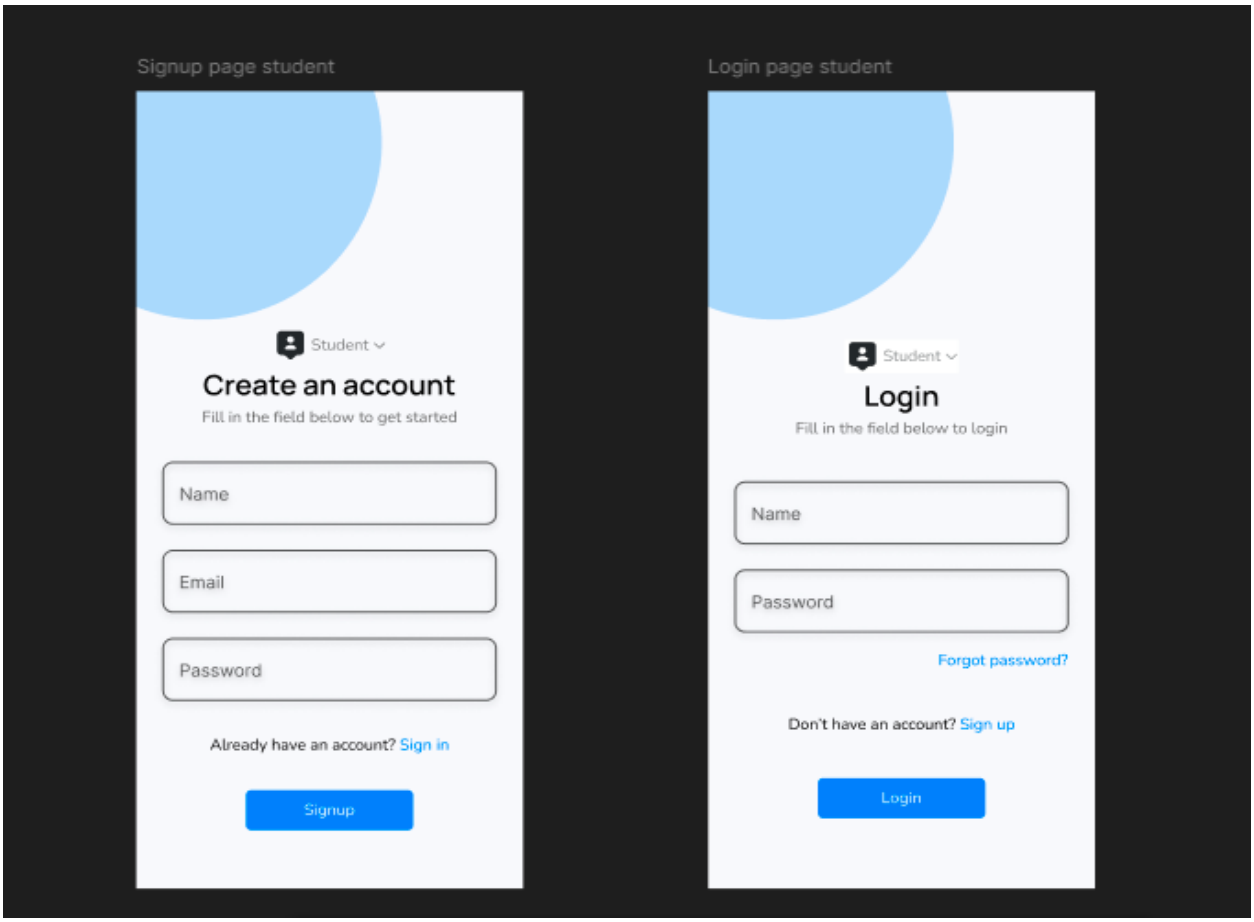


Fig: Students Signup and Login Pages

1.7.2.2. Home Page

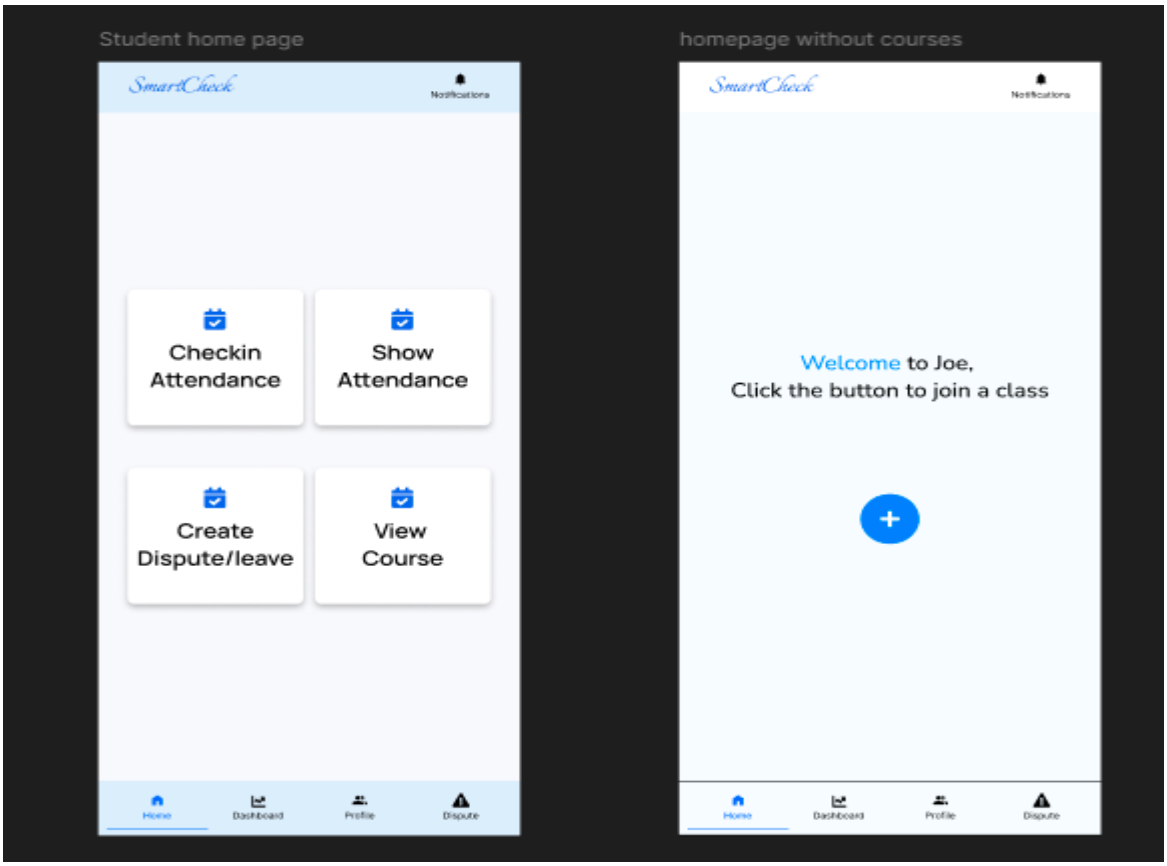
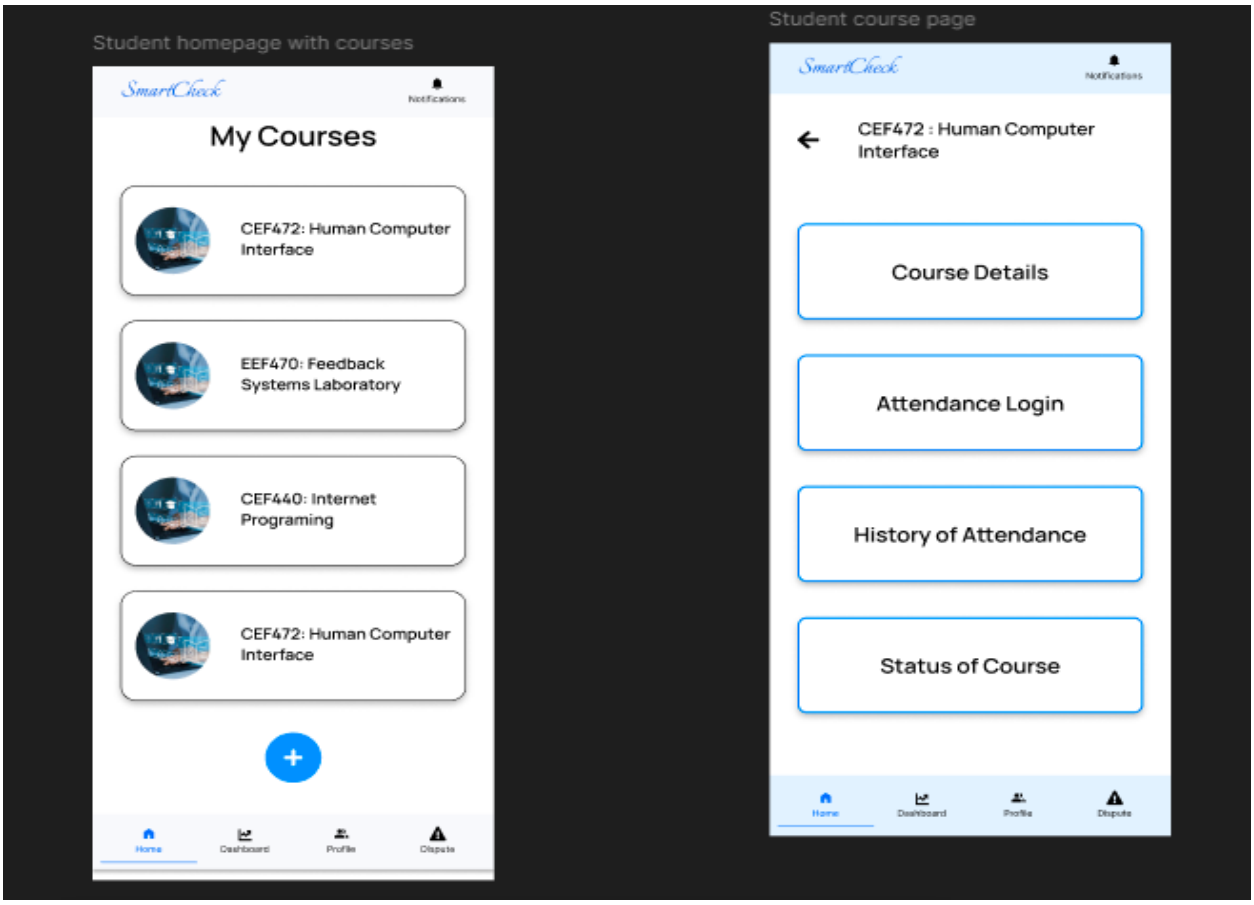


Fig: Student Home page and Course Join page for First time users

1. Home Screen

- **Function**: Acts as the dashboard after login for students.
- **Features**:
    - Quick navigation cards for Check-in, Course View, Attendance History, and Dispute Form.
    - Integrated header with notification alerts.
    - Footer with tab-based navigation (Home, Dashboard, Profile, Dispute).
- **Purpose**: Centralizes all student interactions for fast access and improved usability.

1.7.2.3. Course Page



1.7.2.4. Check-In Page

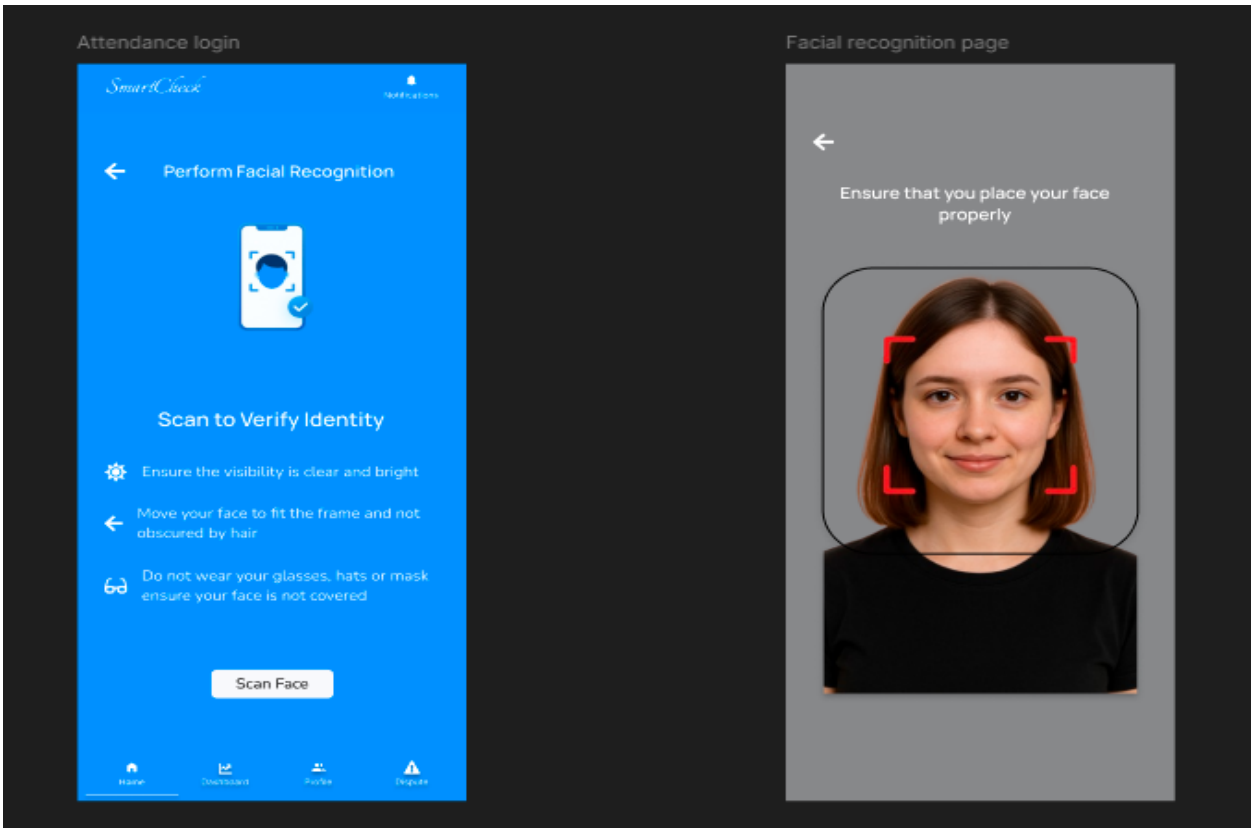


Fig: Attendance Check-in pages for Facial Recognition

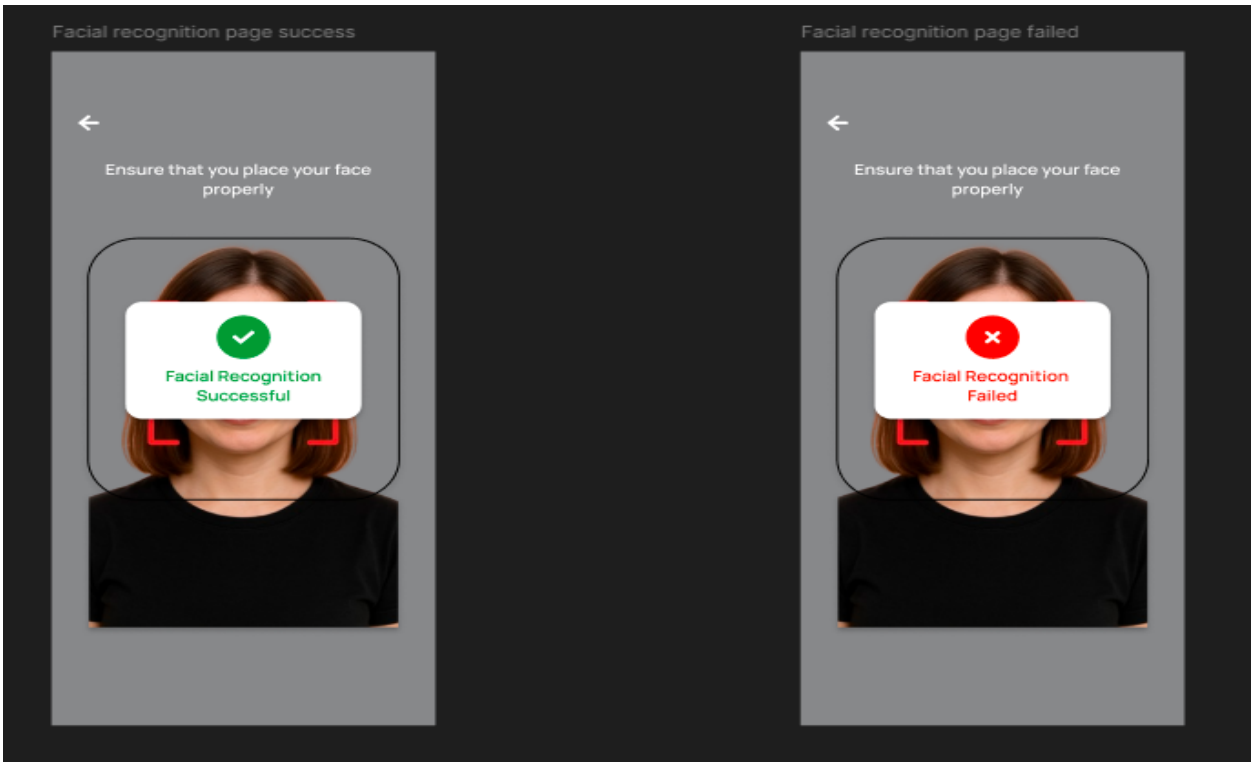
1.7.2.5. Attendance Login Screen

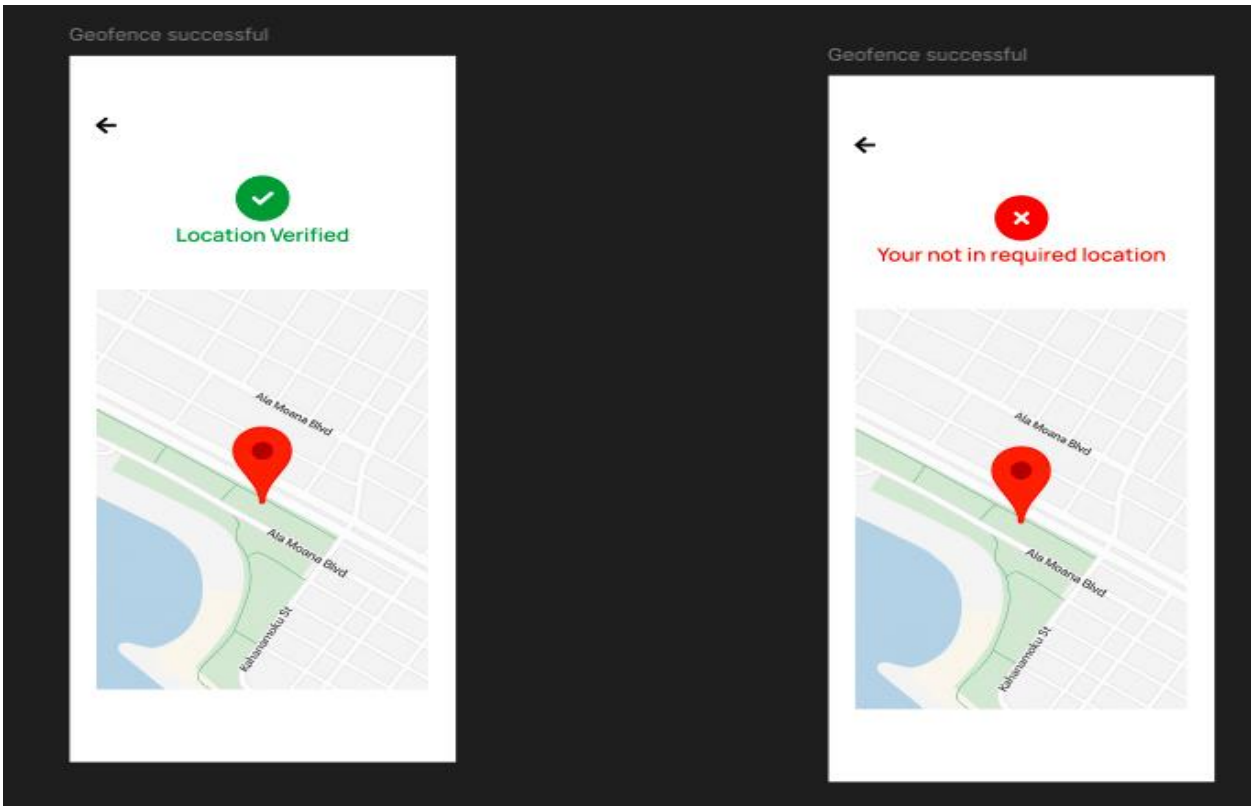
- **Function**: A step before activating the facial recognition scanner.
- **Features**:
  - Displays check-in instructions (lighting, face visibility, etc.)
  - Includes a preview button for proceeding to the face scan.
- **Purpose**: Ensures that users are ready to undergo facial verification and geolocation check before initiating the process.

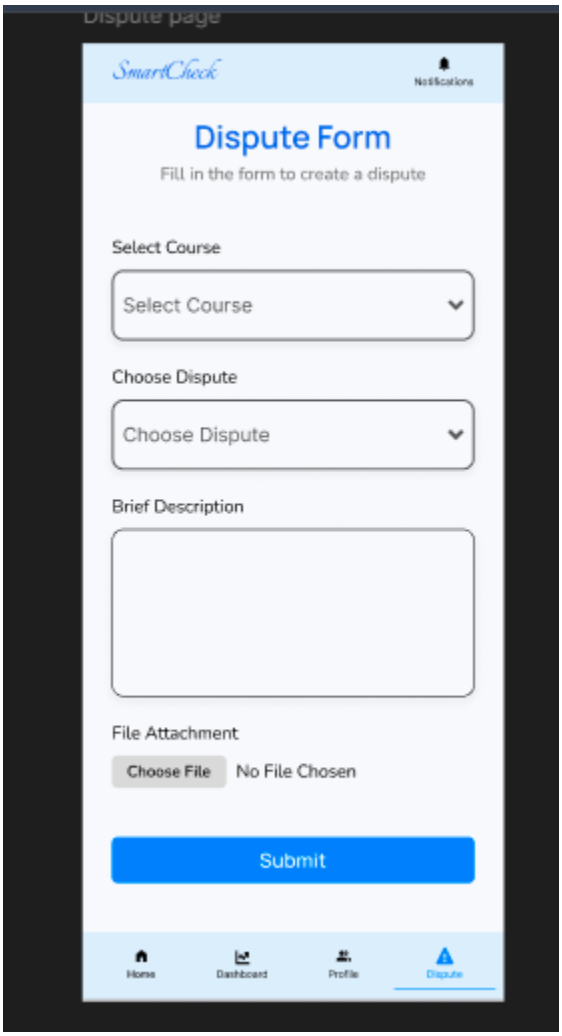
1.7.2.6. Facial Recognition Screen

- **Function**: Handles real-time facial capture and verification.
- **Features**:
  - Uses device's front camera (via react-native-camera).
  - Face frame to assist user positioning.
  - On capture, data is sent for validation.
- **Purpose**: To provide biometric assurance that the student is physically present and matches the registered identity.

1.7.2.7. Facial Recognition and Geofencing Success and Failure Pages







1.7.2.8. Dispute Form Screen

- **Function**: Enables students to request leave or challenge an incorrect attendance log.
- **Features**:
    - Text inputs for reason, date, and optional proof.
    - Submit button sends request to the backend (Firebase or mock database).
- **Purpose**: Establishes a feedback channel between students and instructors to handle exceptions.

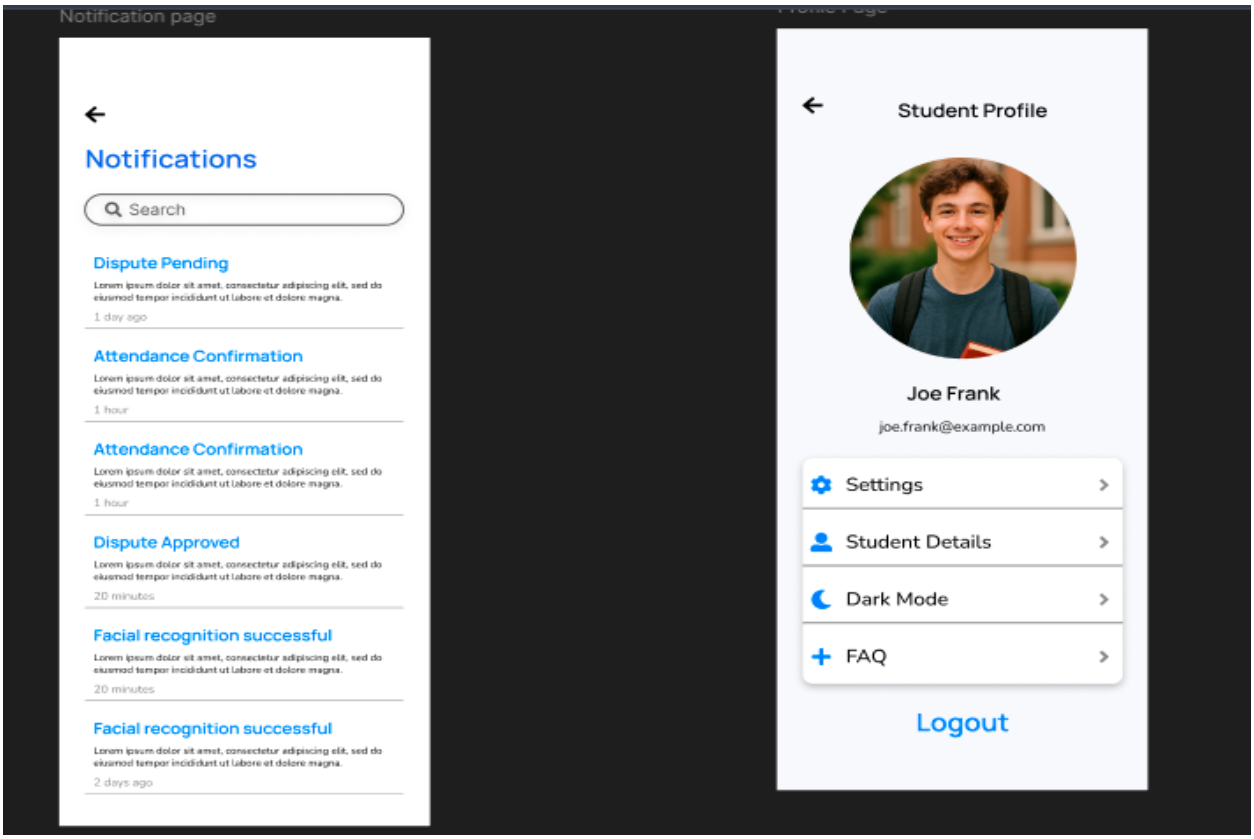


Fig: Notification and Profile Page

1.8. Conclusion

The frontend design and implementation of the SmartCheck application successfully translate complex attendance workflows into a **simple, intuitive, and reliable mobile experience**. Through the use of modular components, consistent visual design, and responsive layouts, the app delivers on its goals of **efficiency, innovation, and integrity**. By leveraging React Native and modern UI principles, SmartCheck is well-prepared to support secure, geofenced, and biometric attendance tracking in real-world academic environments.

1.9. References

- Figma_Designs: https://www.figma.com/design/LFGEhXuGMhYmosJqbLoZoU/MBAMS?node-id=0-1&t=6B5L24QiygPyp3QU-1
- Prototype: https://www.figma.com/proto/LFGEhXuGMhYmosJqbLoZoU/MBAMS?node-id=66-214&p=f&t=6B5L24QiygPyp3QU-0&scaling=scale-down&content-scaling=fixed&page-id=0%3A1&starting-point-node-id=35%3A189&show-proto-sidebar=1